

01_host_baseline_evidence

Host Baseline — Captured Evidence (§11.4.5 / §11.4.6)

Captured	2026-07-06, this session, read-only (no power/control writes — §11.4.133/§12)
Purpose	Ground the HelixLLM programme in ACTUAL host facts, not assumptions
Revision	1

Captured facts (verbatim from `nvidia-smi` / `podman` / `shell`)

GPU

NVIDIA GeForce RTX 5090, driver 570.169, 32607 MiB total, 2 MiB used, 38°C
CUDA Version (driver runtime): 12.8

- Target card confirmed: **RTX 5090, ~32 GB VRAM**, idle at capture (headroom full).
- Driver **570.169 + CUDA 12.8** — CUDA 12.8 is the branch that adds Blackwell **sm_120** support ⇒ RZ-01 de-risked to a build/passthrough-config problem, not a compatibility unknown.
- Power limit 600 W; idle 21 W, 38 °C — thermal/power headroom present; §11.4.133 target-safety must cap sustained load with captured thermal evidence.

CUDA toolkit

- `nvcc` **ABSENT** — no CUDA toolkit installed. Building the vendored `llama.cpp` for `sm_120` requires either installing the CUDA toolkit or compiling inside a CUDA devel container. **[G-HOST-2]**

Container runtime (§11.4.161)

- `podman` 5.7.1, **rootless = true** — §11.4.161 satisfied at runtime level.
- `nvidia-ctk` **ABSENT**, no CDI specs in `/etc/cdi` or `/var/run/cdi` — **NVIDIA Container Toolkit not installed**, so rootless-podman GPU

passthrough is not yet possible. Requires install + `nvidia-ctk cdi generate`. **[G-HOST-1]** (blocks any containerized GPU workload — Phase-1 prerequisite).

Existing serving binaries

- `llama-server`, `llama-cli`, `ollama`, `vllm` — none on PATH.
- `dependencies/LLama_CPP` contains the **llama.cpp source tree** (`CMakeLists.txt`, `cmake/`, `app/`, ...) — **vendored but NOT built**. **[G-HOST-3]** build for Blackwell is a Phase-1 task.

CPU / RAM

64 CPU cores | 251 GiB RAM total, 223 GiB available

- Large headroom: CPU-offload fallback, many concurrent container services, generous page cache for model weights.

Derived setup prerequisites (feed Phase-1 setup scripts + stream 09)

ID	Prerequisite	Blocks	Note
G-HOST-1	Install NVIDIA Container Toolkit + <code>nvidia-ctk cdi generate</code>	ALL rootless-podman GPU workloads	verify rootless CDI passthrough with a real <code>nvidia-smi-in-container</code> proof (\$11.4.69)

G-HOST-1 / V-03 RESOLVED (2026-07-06, captured): OS = ALT Workstation 11.1 (altlinux). `nvidia-container-toolkit 1.18.2-alt1` **IS available in the configured ALT repos** (Sisyphus) — plain `apt-get install`, NO from-source / NO NVIDIA rpm repo needed (V-03 was the top UNCONFIRMED). `libnvidia-container1` / `-tools` also available. `/dev/nvidia{0,ctl,-modeset,-uvm,-uvm-tools}` device nodes present + world-readable (`crw-rw-rw-`); driver 570.169; `libcudart-12.8.1` installed. NOT yet installed: `libnvidia-container`, `nvidia-container-*` binaries. **The install is the one sudo-gated action; everything after (rootless `nvidia-ctk cdi generate` → `~/config/cdi`, verify) is non-privileged.** Install command (operator, once):

```
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit
```

G-HOST-2 | CUDA build toolchain (toolkit or CUDA-devel container) | building llama.cpp / any-from-source GPU engine | prefer a pinned CUDA 12.8 devel container to avoid host pollution |

G-HOST-3 | Build dependencies/LLama_CPP for sm_120 (-DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=120) | local serving | capture a real inference proof (tok/s + nvidia-smi VRAM delta) — not a build-success-only claim (§11.4.108) |

Honest boundary (§11.4.6)

These are point-in-time captured facts. The exact llama.cpp CUDA build flags for Blackwell, and whether vLLM/SGLang have stable sm_120 wheels, are being verified by research stream 01 (cited sources) — do not assume; confirm before the Phase-1 build.